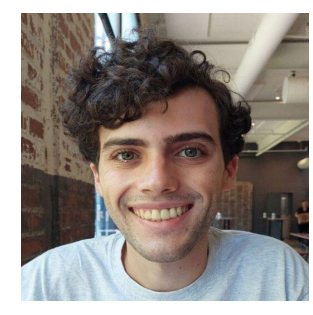




Eli Baum



Sam Buxbaum



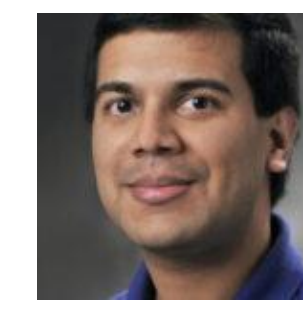
Nitin Mathai



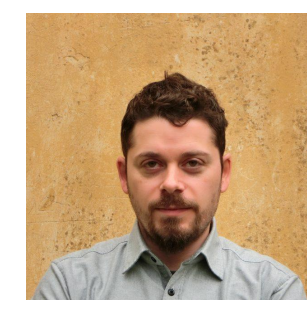
Muhammad Faisal



Vasiliki Kalavri



Mayank Varia



John Liagouris



Github



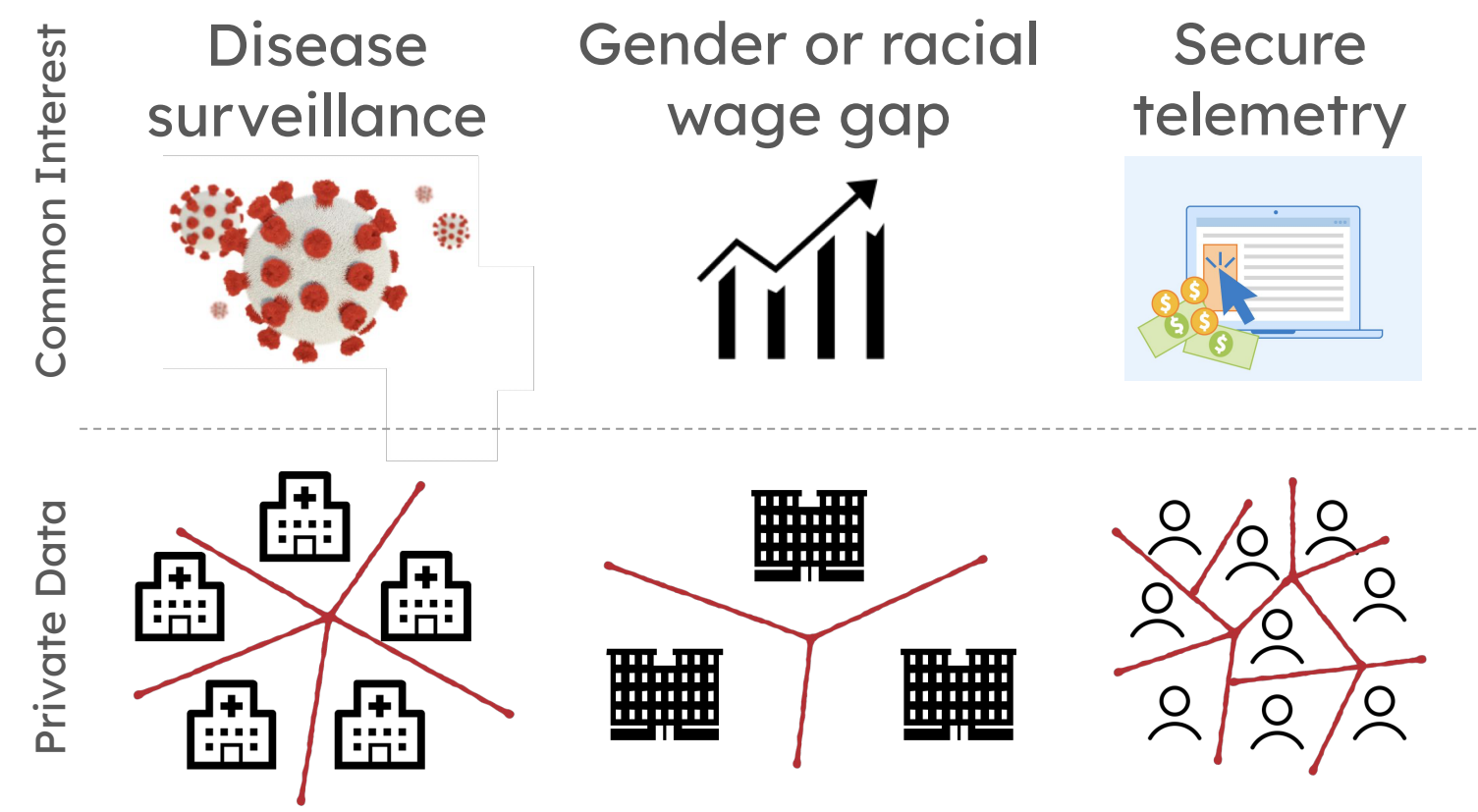
CASPLab



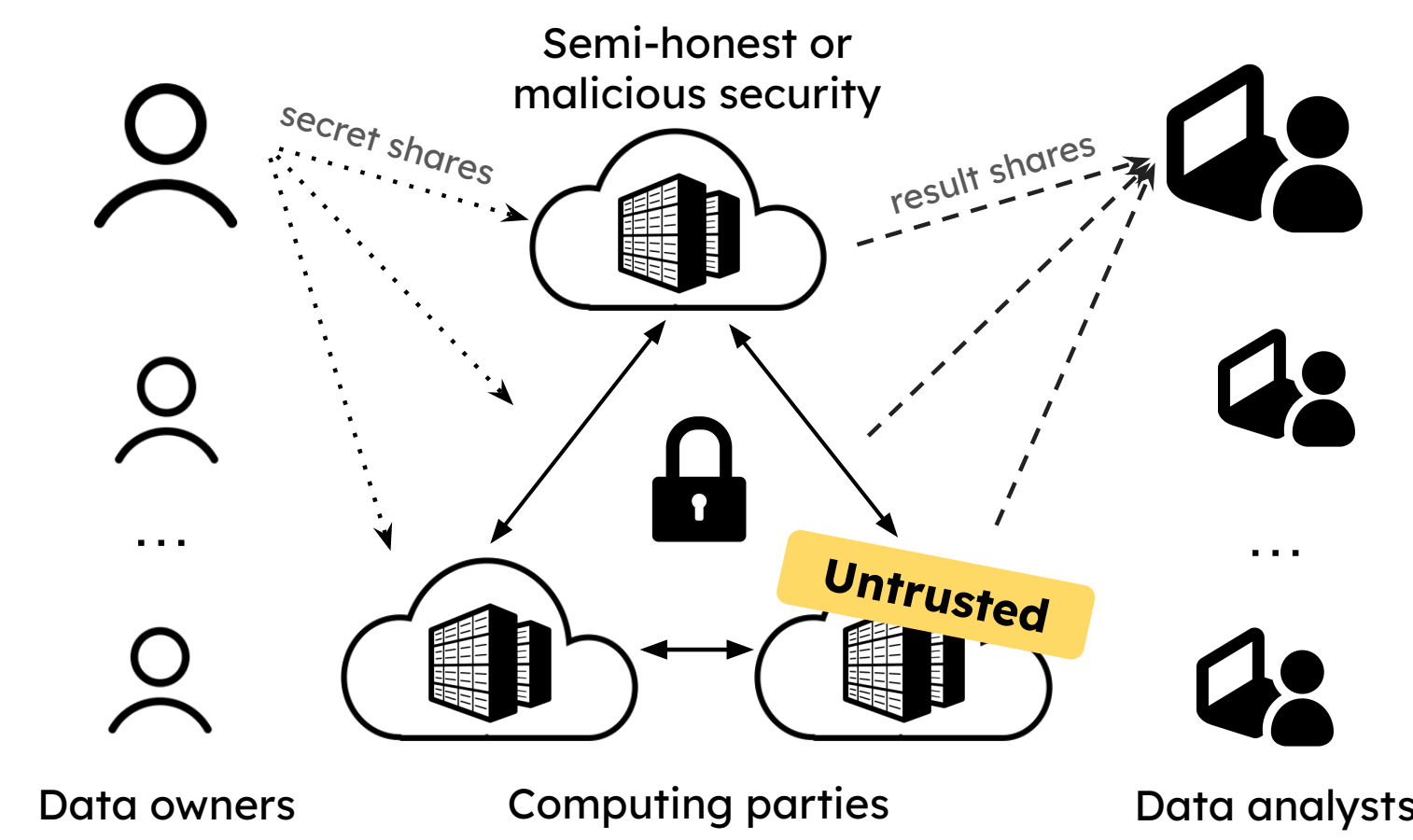
Paper

Protecting Data “In Use” Requires Worst-Case Outputs

Secure collaborative analytics involve joining private data from multiple sources

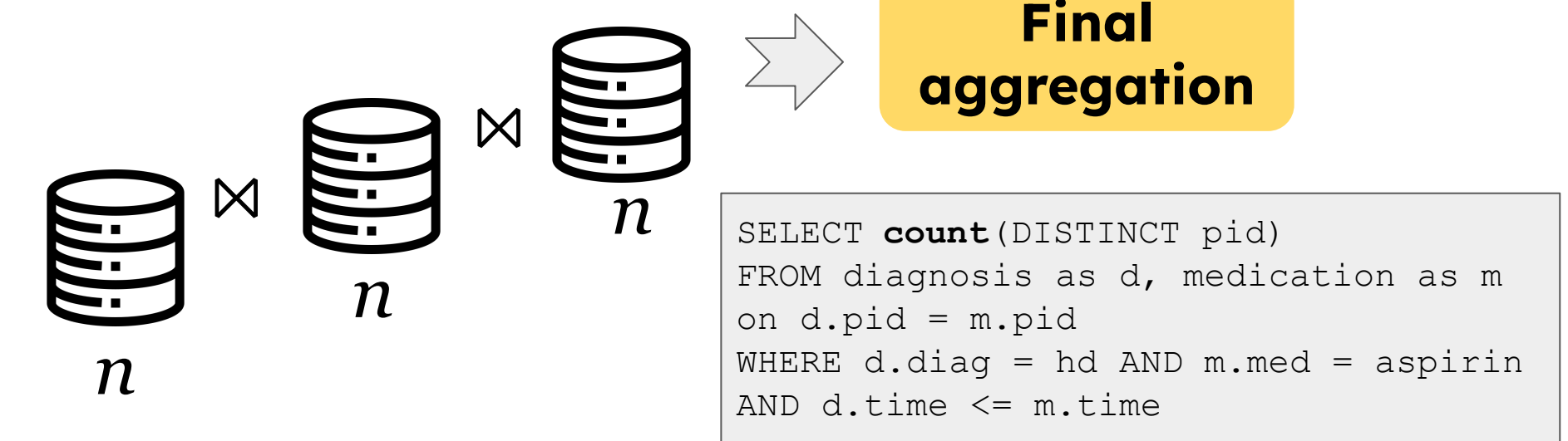


Secure Multiparty Computation (Outsourced Setting)



Can we efficiently evaluate complex analytics without compromising security?

Yes! Collaborative analytics typically return aggregated results:



Problem

- A secure join has $O(n^2)$ output
- k joins $\Rightarrow O(n^{k+1})$ time & space complexity
- Hard in general!

Current Approaches

- Materialize **Cartesian product** of joins
- Require **manual upper bounds** on the join output
- Leak size** of intermediate or output data

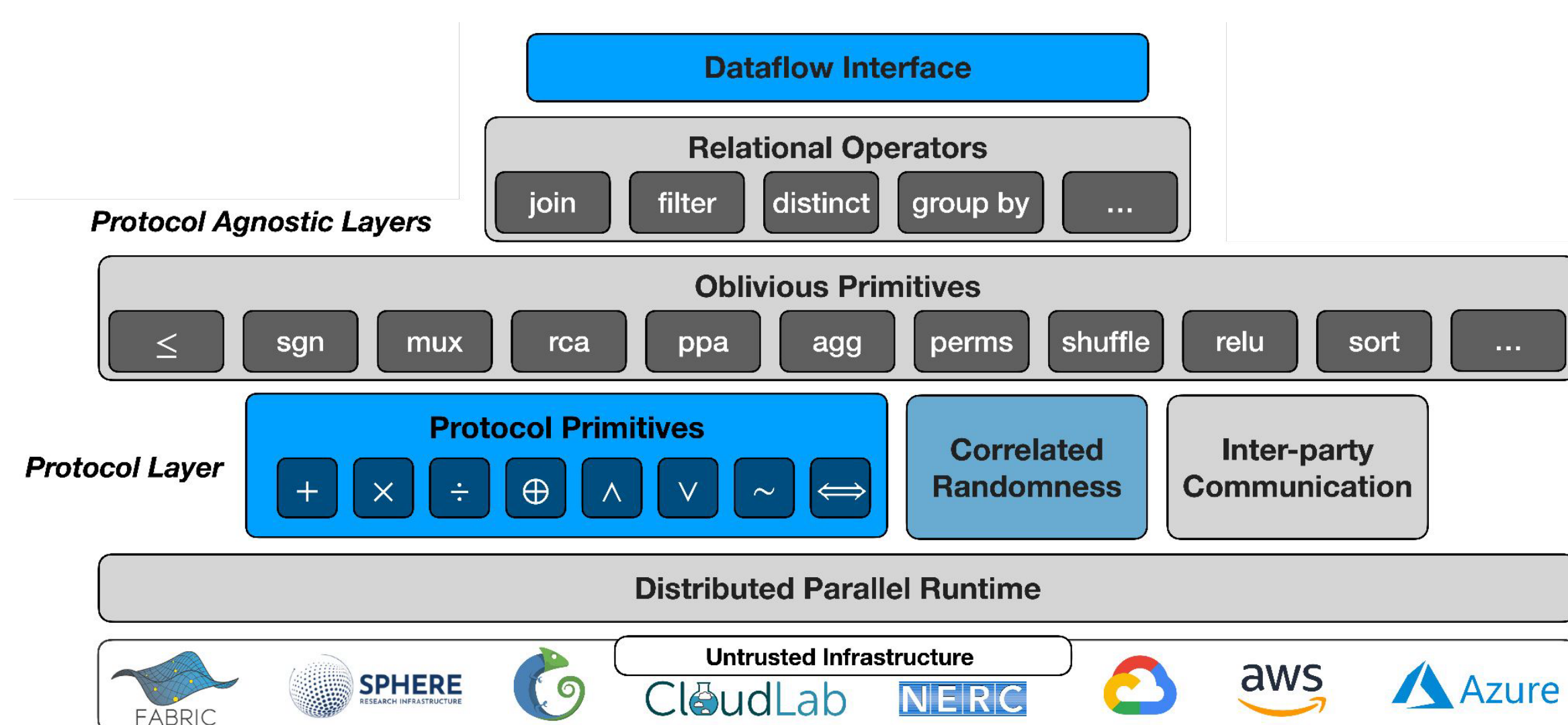
Key Insights

- Real-world queries produce results whose size is worst-case bounded by the input size
- Joins are **followed by aggregations** that are usually amenable to partial evaluation

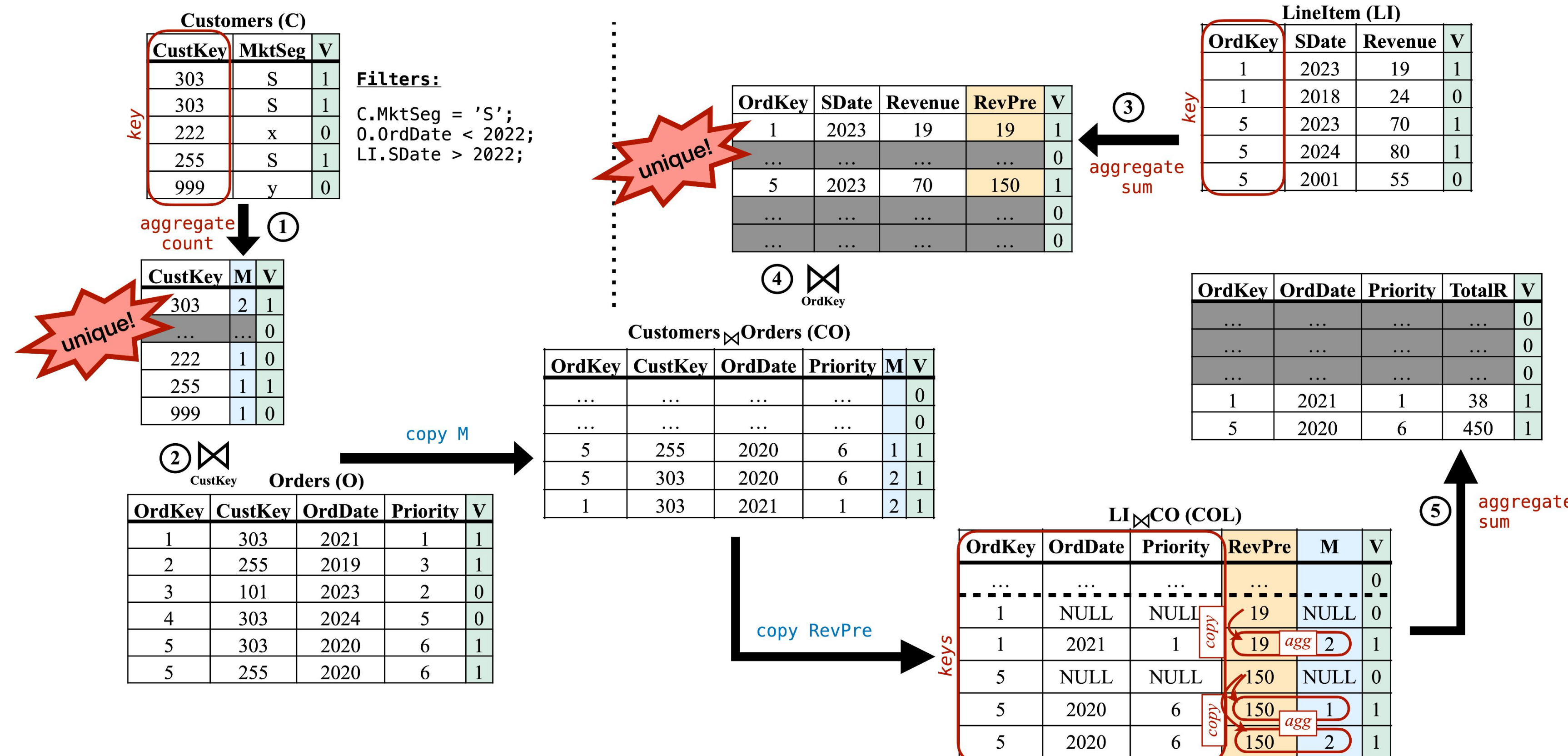
ORQ: An Oblivious Relational Query Engine

A new software stack for relational analytics using secure multiparty computation

Secure query evaluation is reduced to a series of sorts + prefix aggregation primitives

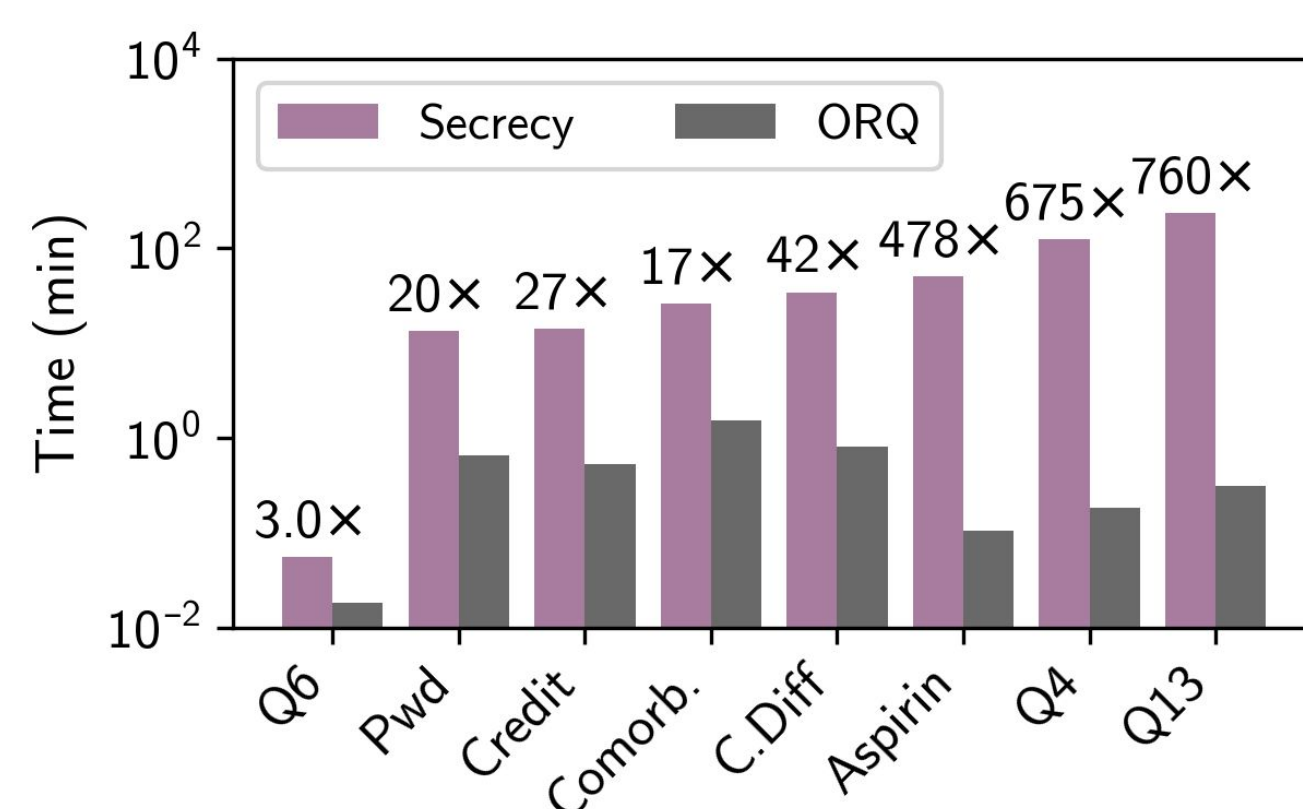


- Strong security:** Designed to run on untrusted infrastructure
- Expressivity:** Complex, multi-way join-aggregation queries
- Generality:** Multiple threat models and security guarantees
- Performance:** Parallelism, vectorization, & custom communicator



ORQ in Action

Comparison with Secrecy (NSDI'23)

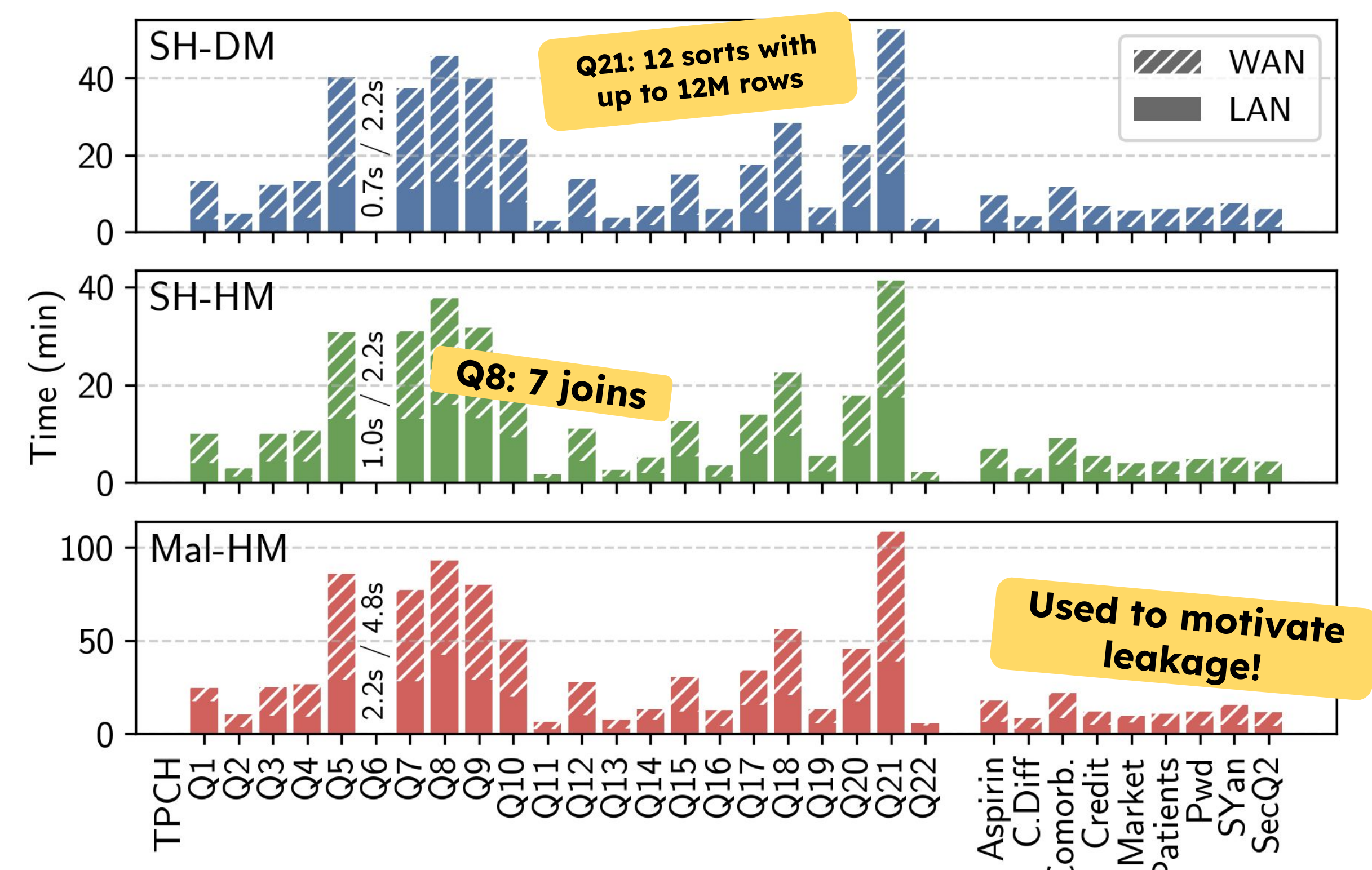


Familiar Dataflow API

```
1. // Initialize C (Customers), O (Orders), LI (Lineitem)
2. ...
3.
4. // Apply filters
5. C.filter("MktSegment" == SEGMENT);
6. O.filter("OrdDate" < DATE);
7. LI.filter("ShipDate" > DATE);
8.
9. // Calculate the revenue
10. LI["Revenue"] = LI["Price"] * (100 - LI["Discount"]) / 100;
11.
12. // Joins and group-by with aggregation
13. auto RES = C.inner_join(O, {"CustKey"})
14.   .inner_join(LI, {"OrdKey"},
15.     {"OrdDate", "OrdDate", copy},
16.     {"Priority", "Priority", copy})
17.   .aggregate({"OrdKey", "OrdDate", "Priority"},
18.     {"Revenue", "TotalRevenue", sum})
19.   .project({"OrdKey", "TotalRevenue",
20.     "OrdDate", "Priority"})
21.   .sort({"TotalRevenue", DESC},
22.     {"OrdDate", ASC})
23.   .limit(10);
```

- 31 queries** with multiple joins and aggregations:
 - Entire TPC-H benchmark at SF1 and SF10 (up to 85M rows, 100M+ intermediate rows)
 - All queries from prior work
- 3 state-of-the-art MPC protocols with different security guarantees (including malicious)
- Asymptotic cost $O(n \log n)$, equal to that of sorting the input tables
- ORQ scales well:** With 10x larger inputs, only ~10x overhead

Performance on complex analytics



Obliviously sorting 500M elements

